Problem Statement

"Modern web applications are increasingly targeted by sophisticated, AI-driven 'Zero-Day' attacks that bypass traditional, rule-based Web Application Firewalls (WAFs). Traditional WAFs rely on static regex patterns and manual rule updates, leading to a high rate of 'false negatives' (missed attacks) and 'false positives' (blocking legitimate users). There is a critical need for an autonomous security gateway that can intercept traffic in real-time, analyze intent using neural networks, and 'heal' itself by automatically generating new defense rules to counter emerging threats without human intervention."

Detailed Project Description

The Goal

To build a **High-Performance Adaptive Security Gateway** that acts as a reverse proxy. It will combine the speed of **C/C++** for traffic interception with the intelligence of **Large Language Models (LLMs)** and **Neural Networks** for threat diagnosis and rule generation.

The System Architecture (The "Brain-Muscle" Model)

The project is divided into two distinct layers that communicate via **System V IPC (Shared Memory)** to ensure near-zero latency.

Layer	Component Name	Technical Role
The Muscle	**C++ Interceptor**	A high-speed reverse proxy (based on Nginx or a custom Socket server) that captures every incoming HTTP request.
The Brain	**AI Inference Engine**	A Python-based service running a **Neural Network (CNN/Transformer)** to score every request for malicious intent.
The Healer	**Agentic LLM**	A local LLM (like Llama-3) that analyzes "Anomaly" logs and writes new C++ regex rules to block the detected attack pattern.

The "Self-Healing" Workflow

This is the core innovation of your project. It follows a continuous loop:

1. **Monitor:** The C++ Interceptor captures a request and places it in **Shared Memory**.
2. **Analyze:** The AI Inference Engine reads the request and calculates a "Threat Score."
3. **Detect:** If the score is high but no existing rule exists (a **Zero-Day**), the request is flagged as an "Anomaly."
4. **Diagnose:** The **Agentic LLM** examines the anomalous request (e.g., a strange ${jndi:ldap...} string).
5. **Heal:** The LLM automatically generates a new **Regex Rule** (e.g., ^.*\$\{jndi:.*$) and pushes it into the C++ Interceptor's active memory.
6. **Verify:** The C++ Interceptor now blocks all similar future requests instantly without needing to ask the AI again.

Technical Requirements for Your Team

Domain	Required Skills / Tools
Systems (C++)	**Socket Programming**, **Multithreading**, **System V IPC (shmget, msgsnd)**, **libpcap**.
AI/ML (Python)	**PyTorch/TensorFlow**, **Ollama** (for local LLMs), **Scikit-learn** (for anomaly detection).
DevOps/Backend	**Docker** (for sandboxing), **REST APIs**, **Nginx Configuration**, **Linux Security**.

Success Metrics

- **Latency:** The AI analysis must add less than **30ms** to the total request time.
- **Accuracy:** Achieve a **>95% Detection Rate** on the OWASP Top 10 attacks.

- **Autonomy:** The system should successfully generate and apply a valid blocking rule for a "Log4j" style injection within **60 seconds** of the first detection.

Roles and Roadmap:

1. **Systems & Backend:**

Phase	Duration	Weekly Milestones	Key Deliverables
Foundation	Weeks 1-2	Set up **Nginx** as a Reverse Proxy. Develop a **C++ module** (ngx_http_module) to intercept request headers and body.	Functional Proxy + Payload Interceptor.
Data Bridge	Weeks 3-4	Implement **System V Shared Memory** (shmget). Design a circular buffer to hold captured HTTP payloads for AI processing.	Low-latency **IPC Bridge** header.
Blocking logic	Weeks 5-6	Implement the "Allow/Deny" logic. Create a **Message Queue** listener; if the AI flags a request, the C++ module drops the connection.	Real-time Blocking Service.
Optimization	Weeks 7-8	Implement **Semaphores** to handle concurrent worker processes. Benchmark latency (aim for <25ms overhead).	Production-ready Backend.

2. **AI & Machine Learning Engineer:**

Phase	**Duration**	**Weekly Milestones**	**Key Deliverables**
Data Prep	Weeks 1-2	Clean and vectorize the **CSIC 2010 HTTP Dataset**. Extract features (length, character entropy, special character ratio).	Training-ready data pipeline.
Model Build	Weeks 3-4	Train a **DistilBERT** or **Random Forest** model for binary classification (Normal vs. Attack). Quantize to **ONNX** for speed.	Optimized Inference Model.
Inference API	Weeks 5-6	Build a Python worker that maps the **Shared Memory** created by the Systems Lead and runs inference on incoming buffers.	High-speed Inference Service.
Refinement	Weeks 7-8	Fine-tune the model to reduce false positives. Implement a "Shadow Mode" to verify accuracy against live traffic.	Final AI Engine.

3. **Security & Automation:**

Phase	**Duration**	**Weekly Milestones**	**Key Deliverables**
Setup	Weeks 1-2	Configure **Ollama** locally with **Llama-3-8B**. Create a library of "True" regex patterns for OWASP Top 10.	Local LLM Environment.
Agent Logic	Weeks 3-4	Build the "Diagnosis Agent" that takes anomalous request strings and identifies the specific attack type (e.g., SQLi, Log4j).	Attack Diagnosis Agent.
Healing Loop	Weeks 5-6	Develop the **Rule Generator**: The Agent writes a new C++ regex or iptables rule and pushes it to the Systems Lead's module.	Automated Healing Script.
Red Teaming	Weeks 7-8	Launch simulated attacks (SQL Map, Fuzzing). Verify the system "heals" (generates a rule) within 60s of the first exploit.	Security Validation Report.